arXiv:submit/7511373 [cs.DC] 23 Apr 2026

DECIDABILITY OF LIVELOCK DETECTION FOR PARAMETERIZED SELF-DISABLING UNIDIRECTIONAL RINGS

A PREPRINT

Aly Farahat
Independent Researcher
aly.farahat@perpetual-dynamics.com

ABSTRACT

We revisit the decidability of livelock detection in self-disabling unidirectional ring protocols, a problem shown to be undecidable in general by Klinkhamer and Ebnenasir via reduction from the periodic domino problem. Despite this, practical protocols routinely admit finite proofs of livelock freedom via the same tiling constraints, and synthesis of parameterized self-stabilizing rings has been shown to be decidable—suggesting a gap between theory and practice.

We identify the source of this gap: the apparent unboundedness of livelock reasoning is an artifact of working in the transition space. By lifting to an *equivariant product space*—the space of transition–witness pairs coupled by the zigzag equivariance conditions of Farahat—we show that self-disabling induces a structure in which closure and period are preserved under backward propagation. This yields a *bounded witness property*: every livelock, constructed from a set of local transitions T, admits a representative as a local cycle of length at most $|T|^2$ in a finite product graph, independent of the ring size.

We derive a sound and complete decision procedure via greatest fixed-point iteration on the product graph. Our results demonstrate that decidability emerges not by restricting the problem syntactically, but by exposing its underlying finite combinatorial structure. We validate on over 4,300 protocols with zero errors, extend to $(1, 1)$-asymmetric protocols, and derive a circulation law classifying livelocks by ring size. Code and algebraic foundation are at `https://github.com/cosmoparadox/mathematical-tools`.

***K*eywords** livelock detection · self-stabilization · parameterized verification · unidirectional ring · fixed point

1 Introduction

Self-stabilization [6] guarantees that a distributed system recovers from arbitrary transient faults to a legitimate configuration, regardless of the initial state. A key obstacle to self-stabilization in ring protocols is *livelock*: an infinite execution in which processes continually change state but never reach legitimacy. Determining whether a given protocol admits a livelock—for any ring size—is a fundamental parameterized verification problem.

The tension. Klinkhamer and Ebnenasir [15] established that livelock detection for parameterized rings of self-disabling processes is Π^0_1-complete, via a reduction from the periodic domino problem. Yet practical protocols routinely admit finite proofs of livelock freedom [13], and the *synthesis* of self-stabilizing rings—ostensibly harder than verification—has been shown to be decidable [15]. This suggests a mismatch between the theoretical barrier and the structure actually present in self-disabling protocols.

The resolution. The obstruction is not the behavior itself, but the representation. In the transition space, livelock reasoning searches over walks of unbounded period—an infinite domain. But in the *equivariant product space*—the space of transition–witness pairs coupled by the zigzag conditions of Farahat [10, 11]—the structure becomes finite. Self-disabling ensures that the coupling between adjacent processes is one-to-one (the token propagation discipline),

which forces equivariant walks to have equal length and closure to propagate. The product space has at most $|T|^2$ elements. Every valid livelock structure appears as a cycle in this finite space.

Our contribution. We construct the *product transition graph* $G_\times(T)$ whose nodes are transition–witness pairs and whose arcs encode the four equivariance conditions—faithfully and without additional restrictions. We prove:

1. *Closure propagation:* equivariant walks inherit closure and period from the transition walk they witness (Lemma 2).
2. *Bounded witness:* a protocol admits a livelock if and only if T contains a non-empty *witness-closed* subset, whose cycles have length at most $|T|^2$ (Theorem 5).
3. *Backward propagation:* every cycle in a witness-closed product graph propagates backward to a new cycle of the same length, with chaining forced by a shared boundary value in the zigzag (Lemma 4).
4. *Decision procedure:* the greatest witness-closed subset $L^* \subseteq T$ is computable via monotone fixed-point iteration on the product graph; a livelock exists if and only if $L^* \neq \emptyset$ (Algorithm 1).

Our construction does not assume determinism, handling non-deterministic protocols—a strictly broader class than previously studied. We extend to $(1,1)$-asymmetric protocols covering Dijkstra's token ring, and derive a *circulation law* classifying livelock structures by ring size and number of enabled processes.

Our approach instantiates a recurring pattern in verification: problems that appear unbounded in their native representation become finite when lifted to a space where the governing invariant is explicit (Section 9).

2 Definitions

2.1 Unidirectional Ring Protocols

A *unidirectional ring* of size K consists of processes $P_0, P_1, \ldots, P_{K-1}$ arranged in a directed cycle. Each process P_i maintains a state variable $v_i \in \mathbb{Z}_m = \{0, 1, \ldots, m-1\}$ and can read v_i and $v_{i-1 \bmod K}$.

Definition 1 (Transition)**.** *A* transition *is a triple $t = (p, o, w) \in \mathbb{Z}_m^3$, meaning: if the predecessor's value is p and the process's own value is o, the process may write w. We call p the* pred value*, o the* own value*, and w the* written value*.*

Definition 2 (Self-disabling)**.** *A protocol T is* self-disabling *if for every transition $(p, o, w) \in T$, there is no transition $(p, w, u) \in T$ for any u. That is, after a process fires (p, o, w), it cannot fire again while the predecessor's value remains p—it must wait for an intervening state change from the predecessor. Throughout this paper, all protocols are self-disabling.*

Definition 3 (Protocol)**.** *A* protocol *is a finite self-disabling set $T \subseteq \mathbb{Z}_m^3$. A transition $(p, o, w) \in T$ is* enabled *at process P_i in configuration $(v_0, \ldots, v_{K-1})$ if $v_{i-1} = p$ and $v_i = o$.*

Definition 4 (Livelock)**.** *A* livelock *is a global periodic execution of a ring: K processes fire transitions indefinitely, returning the entire ring to the same global configuration after N steps, without ever reaching a legitimate state. Equivalently, a livelock is a valid tiling of a $K \times N$ torus, where each row is a closed walk of* local *transitions executed by one process, and adjacent rows satisfy the equivariance conditions (Section 3). The walk length N is the* local period*; the number of processes K is the* ring size*.*

2.2 The (l, q)-Asymmetry Model

Definition 5 ((l, q)-asymmetry)**.** *A ring protocol has (l, q)-asymmetry if there are l* distinguished *process types (each appearing exactly once) and q* other *types (each appearing one or more times, filling the remaining $K - l$ positions).*

The key cases are:

- $(0, 1)$**-asymmetry (symmetric):** All processes use the same transition set T. This is the primary focus of this paper.
- $(1, 1)$**-asymmetry:** One distinguished process P_0 uses T_0; all others use T_o. This covers Dijkstra's token ring. We extend our framework to this case in Section 8.
- **General** (l, q)**:** Stated as a future direction.

2.3 The H-Graph

Definition 6 (Shadow and witness)**.** *The* shadow *of a consecutive transition pair (t_i, t_j) with $\mathrm{own}(t_j) = \mathrm{wr}(t_i)$ is $\sigma(t_i, t_j) = (\mathrm{pred}(t_i), \mathrm{pred}(t_j))$. A transition u* witnesses *shadow (a, b) if $\mathrm{own}(u) = a$ and $\mathrm{wr}(u) = b$.*

Definition 7 (H-graph)**.** *Given a transition set L, the* H-graph $H(L)$ *is a directed graph with vertex set L and an edge $t_i \to t_j$ (an* H-edge*) whenever* $\mathrm{own}(t_j) = \mathrm{wr}(t_i)$*. A closed walk in $H(L)$ is a* pseudolivelock*: a sequence of local transitions that a single process could execute cyclically, if appropriately enabled by its predecessor. A pseudolivelock is necessary but not sufficient for a global livelock—it captures the local structure without guaranteeing the existence of consistent witnessing across the ring.*

3 The Product Transition Graph

3.1 Equivariance Conditions

In a livelock on a ring of size K, each process P_i executes a closed walk $W^{(i)}$ of length N in the H-graph. Adjacent processes are coupled through a *zigzag*: at each time step, the predecessor's transition both enables the current step (through its pre-fire state) and enables the next step (through its written value). This coupling is captured by four *equivariance conditions*.

For a product graph edge $(t_k, w_k) \to (t_{k+1}, w_{k+1})$, the connecting witness w_{k+1} links t_k to t_{k+1} in the zigzag $\ldots w_k \to t_k \to w_{k+1} \to t_{k+1} \to w_{k+2} \ldots$:

1. $\mathrm{own}(w_{k+1}) = \mathrm{pred}(t_k)$: w_{k+1}'s pre-fire state is the value t_k reads from its predecessor.
2. $\mathrm{wr}(w_{k+1}) = \mathrm{pred}(t_{k+1})$: w_{k+1}'s written value is what t_{k+1} reads from its predecessor.
3. $\mathrm{wr}(t_k) = \mathrm{own}(t_{k+1})$: H-edge in the t-walk.
4. $\mathrm{wr}(w_k) = \mathrm{own}(w_{k+1})$: H-edge in the w-walk.

Remark 1 (Emergent shadow witness)**.** *Conditions (1) and (2) together show that w_{k+1} witnesses the shadow $\sigma(t_k, t_{k+1})$:* $\mathrm{own}(w_{k+1}) = \mathrm{pred}(t_k)$ *and* $\mathrm{wr}(w_{k+1}) = \mathrm{pred}(t_{k+1})$*. The shadow witness emerges directly from the zigzag structure. Combining conditions (1) and (4) gives* $\mathrm{wr}(w_k) = \mathrm{pred}(t_k)$*, which serves as the node condition for the product graph.*

3.2 Construction

Definition 8 (Product transition graph)**.** *Given a transition set T, the* product transition graph $G_\times(T)$ *is defined as follows.*

Nodes: *Pairs* $(t, w) \in T \times T$ *such that* $\mathrm{wr}(w) = \mathrm{pred}(t)$.

Edges: $(t_k, w_k) \to (t_{k+1}, w_{k+1})$ *if all four equivariance conditions hold.*

The product graph is built once *from T and is not reconstructed during the algorithm.*

Proposition 1 (Size)**.** *$G_\times(T)$ has at most $|T|^2$ nodes and at most $|T|^4$ edges.*

3.3 Backward Propagation

A cycle in $G_\times(T)$ encodes the coupling between two adjacent processes. In a ring, this coupling extends to all K processes through *backward propagation*: the w-walk of one coupling becomes the t-walk of the predecessor's coupling. The product graph captures this structure internally: if an arc has w-pair (w_1, w_2), the predecessor coupling requires an arc with t-pair (w_1, w_2)—which is another arc in the *same* product graph.

This observation motivates the three pruning operations in the algorithm.

4 The Bounded Witness Theorem

The key obstacle to decidability is that regularity is invisible at the level of single transition walks. A local H-cycle may admit multiple equivariant witnesses, and its projection onto the predecessor's walk may be compound. Analyzing walks individually—as in [15]—leads to an unbounded search. The correct object is therefore not an individual walk but an *equivariant pair* of walks: a transition walk coupled with its witness. In this lifted space, the one-to-one zigzag coupling induced by self-disabling becomes explicit, closure propagates from one walk to its witness, and boundedness follows from the finiteness of the pair space.

We develop this characterization through five results, building from pure equivariance to the full decidability argument.

4.1 Equivariant Walks Propagate Closure

Lemma 2 (Closure propagation)**.** *Let $W_t = (t_0, \ldots, t_{N-1})$ be a closed walk in $H(T)$, and let $W_w = (w_0, \ldots, w_{N-1})$ be a walk in $H(T)$ equivariant to W_t—that is, w_k witnesses the shadow $\sigma(t_{k-1}, t_k)$ at every step. Then W_w is also a closed walk of the same length N.*

Proof. The equivariance conditions give H-edges in the w-walk: $\mathrm{wr}(w_k) = \mathrm{own}(w_{k+1})$ for each k (condition 4). At the wrap-around, condition 4 at the closing arc of W_t gives $\mathrm{wr}(w_{N-1}) = \mathrm{own}(w_0)$. Therefore W_w is a closed walk. Since each node of W_t contributes exactly one witness, W_w has the same length N. □

This is the foundational observation: equivariance between a closed t-walk and a w-walk forces the w-walk to close with the same period. No product graph is needed—this is a purely local property of equivariant walks in $H(T)$.

Remark 2 (Self-disabling and the zigzag)**.** *Lemma 2 depends critically on self-disabling. In a self-disabling protocol, a process cannot fire again until its predecessor changes state. This means that between consecutive firings of the predecessor, the successor fires exactly once. The coupling between adjacent processes is therefore one-to-one: each step of the t-walk corresponds to exactly one connecting witness in the w-walk, producing the zigzag $\ldots w_k \to t_k \to w_{k+1} \to t_{k+1} \to \ldots$ with both walks of equal length.*

Without self-disabling, a process could fire multiple times between consecutive predecessor firings. The coupling would no longer be one-to-one—a single t-step could require multiple w-steps, or vice versa—and the walks could have different lengths. Lemma 2 would fail, the product graph would not faithfully capture equivariant walk pairs, and the bounded witness property would not hold.

This one-to-one coupling is the token propagation discipline identified by Farahat and Ebnenasir [11]: in a self-disabling ring, an enabled process (a "token") fires, disables itself, and enables its successor—propagating the token forward without duplication or destruction. The zigzag is the algebraic expression of this discipline, and Lemma 2 is its direct consequence.

4.2 The Product Graph Captures Pairwise Equivariant Walks

By Definition 8, a cycle in $G_\times(L^*)$ is precisely a pair of equivariant closed walks: the t-walk and the w-walk, coupled by the four equivariance conditions at every step. By Lemma 2, both walks are closed and have the same length.

Conversely, any pair of equivariant closed walks of length N over L^* defines a cycle of length N in $G_\times(L^*)$. The product graph is a faithful representation of pairwise equivariant closed walks—it imposes no conditions beyond equivariance.

4.3 Bounded Cycle Length

Lemma 3 (Cycle length bound)**.** *Every simple cycle in $G_\times(L^*)$ has length at most $|L^*|^2 \leq |T|^2$.*

Proof. $G_\times(L^*)$ has at most $|L^*|^2$ nodes. A simple cycle visits each node at most once. □

This bounds the local period of any pairwise equivariant walk pair: $N \leq |T|^2$. The bound is structural—it follows from the finiteness of the node space, not from any property of specific protocols.

4.4 Backward Propagation of Cycles

Definition 9 (Witness-closed set)**.** *A set $L^* \subseteq T$ is* witness-closed *if the product graph $G_\times(L^*)$ satisfies:*

1. ***Cyclicity:*** *$G_\times(L^*)$ contains at least one cycle.*
2. ***Backward closure:*** *For every arc on a cycle with w-pair (w_1, w_2), there exists an arc $(w_1, u) \to (w_2, v)$ on a cycle in $G_\times(L^*)$.*

Lemma 4 (Backward propagation of cycles)**.** *Let L^* be witness-closed. If C is a cycle in $G_\times(L^*)$, then there exists a cycle C' in $G_\times(L^*)$ of the same length, whose t-walk equals C's w-walk.*

Proof. Let C have length N, with t-walk W_t and w-walk W_w. By Lemma 2, W_w is a closed walk of length N.

Backward closure provides, for each w-edge (w_k, w_{k+1}), an arc $(w_k, a_k) \to (w_{k+1}, b_k)$ on some cycle in $G_\times(L^*)$. This arc is part of a pairwise equivariant walk pair whose t-walk passes through w_k and w_{k+1}. The transition b_k is the connecting witness in the zigzag, with $\mathrm{own}(b_k) = \mathrm{pred}(w_k)$ and $\mathrm{wr}(b_k) = \mathrm{pred}(w_{k+1})$.

Define the witness walk $W_u = (u_0, \ldots, u_{N-1})$ by $u_{k+1} = b_k$ (indices modulo N). By Lemma 2 applied to the closed walk W_w and the equivariant walk W_u, the walk W_u is closed with length N. The pair (W_w, W_u) defines a cycle C' in $G_\times(L^*)$ of length N, with t-walk W_w.

To verify that W_u is indeed equivariant to W_w: the connecting witness $u_{k+1} = b_k$ has $\mathrm{own}(b_k) = \mathrm{pred}(w_k)$ and $\mathrm{wr}(b_k) = \mathrm{pred}(w_{k+1})$, satisfying conditions (1) and (2). Condition (3) is inherited from C. Condition (4) follows from the shared boundary value: $\mathrm{wr}(u_k) = \mathrm{wr}(b_{k-1}) = \mathrm{pred}(w_k) = \mathrm{own}(b_k) = \mathrm{own}(u_{k+1})$. □

Remark 3. *Since C' is itself a cycle in $G_\times(L^*)$ and L^* is witness-closed, the lemma applies to C', yielding C'', then C''', and so on. Each cycle's t-walk is the previous cycle's w-walk, mirroring the row-by-row structure of the torus.*

4.5 The Bounded Witness Characterization

Theorem 5 (Bounded Witness Characterization)**.** *A self-disabling unidirectional ring protocol with transition set T admits a livelock if and only if T contains a non-empty witness-closed subset L^*.*

Proof. **Direction 1: every livelock is captured.** A livelock at ring size K is, in the sense of [15], a valid tiling of the $K \times N$ torus. Each row i is a closed walk $W^{(i)}$ of length N in the H-graph, and adjacent rows satisfy the four equivariance conditions. Let L^* be the set of all transitions appearing in the tiling.

Each pair of adjacent rows $(W^{(i)}, W^{(i-1)})$ is a pair of equivariant closed walks, hence defines a cycle of length N in $G_\times(L^*)$. This establishes cyclicity.

For backward closure: consider an arc on the cycle of coupling (P_i, P_{i-1}), with w-pair (w_k, w_{k+1}). One row inward, the coupling (P_{i-1}, P_{i-2}) provides exactly this pair as a t-pair, with the arc on its cycle. The torus wraps after K steps inward. Backward closure is satisfied at every arc.

Direction 2: every witness-closed set captures a livelock. Let L^* be a non-empty witness-closed set. By cyclicity, a cycle C_0 exists in $G_\times(L^*)$. By Lemma 4, C_0 propagates backward to C_1, then C_2, and so on—a chain of cycles, all of the same length $N \leq |T|^2$ (Lemma 3), all in $G_\times(L^*)$.

Since $G_\times(L^*)$ has finitely many cycles of length N, the chain must revisit a cycle: C_j is a rotation of C_i for some $j > i$. The segment $C_i, \ldots, C_{j-1}$ forms an orbit of period $p = j - i$: p equivariant closed walk pairs, each witnessing the next, wrapping around to the first.

This orbit is a valid tiling of the $p \times N$ torus: p rows, N columns, with equivariance between adjacent rows and the torus closing by the orbit property. By [15], this constitutes a livelock at ring size $K = p$. □

Corollary 6 (Decidability)**.** *Livelock detection for self-disabling unidirectional ring protocols is decidable. The product graph $G_\times(T)$ has at most $|T|^2$ nodes and $|T|^4$ arcs. The existence of a witness-closed subset is decidable by computing the greatest such subset via a monotone fixed-point iteration on the finite product graph.*

Remark 4 (Torus dimension bounds)**.** *Both structural parameters of the livelock torus are bounded. The local period satisfies $N \leq |T|^2$ (Lemma 3). For the orbit period p: the backward chain consists of cycles of length N in a graph of at most $|T|^2$ nodes, so $p \leq \binom{|T|^2}{N}$. The total search space is at most $2^{|T|^2} - 1$. The ring size K itself is unbounded—the parameterized question asks whether a livelock exists for* any *K—but the orbit period p determines the minimum ring size at which a livelock occurs. Larger ring sizes are governed by the circulation law (Section 6).*

5 The Algorithm

By Theorem 5, it suffices to compute the greatest witness-closed subset $L^* \subseteq T$. The algorithm does so by building the product graph and iteratively pruning arcs that violate the two witness-closure properties.

The algorithm enforces the two witness-closure properties through three pruning operations:

- **SCC** enforces cyclicity by removing arcs not on any cycle.

Algorithm 1 Livelock Detection for Self-Disabling Unidirectional Ring Protocols

Require: Transition set T (symmetric protocol)
Ensure: LIVELOCK or LIVELOCK-FREE

```
Build product graph G_×(T) with four equivariance conditions (Definition 8)
repeat
    SCC: Remove all arcs not on a cycle                                  ▷ enforce cyclicity
    S_t ← {τ : (τ, w) is on a cycle for some w}                          ▷ t-components
    Square: Remove arcs where w_k ∉ S_t or w_{k+1} ∉ S_t
    Backward: For each surviving arc with w-pair (w_1, w_2):
        if no arc (w_1, u) → (w_2, v) exists in G_× with u, v ∈ S_t, remove the arc
                                                                         ▷ enforce backward closure
until no arcs removed
if G_× has surviving arcs then
    return LIVELOCK                                                      ▷ L* = S_t
else
    return LIVELOCK-FREE
end if
```

- **Square** is an auxiliary step that extracts S_t—the set of transitions appearing as t-components on cycles—and removes arcs whose w-components are not in S_t. This is implied by backward closure (if $(w_1, u) \to (w_2, v)$ is on a cycle, then w_1 and w_2 appear as t-components), but computing it explicitly provides the S_t set needed by the backward closure check and accelerates convergence.
- **Backward closure** checks each surviving arc's w-pair (w_1, w_2): does $G_\times$ contain an arc $(w_1, u) \to (w_2, v)$ with $u, v \in S_t$?

The iteration computes the greatest fixed point. It is monotone (arcs are only removed) and operates on a finite set, so it terminates.

Theorem 7 (Correctness)**.** *Algorithm 1 computes the greatest witness-closed subset $L^* \subseteq T$.*

Proof. **Completeness** (no livelock transitions are lost). By Direction 1 of Theorem 5, any livelock's transitions form a witness-closed set whose arcs are on cycles and satisfy backward closure. Each pruning step preserves these arcs: SCC preserves arcs on cycles, square preserves arcs whose w-components are t-components (every livelock transition serves in both roles), and the backward closure step preserves arcs whose w-edges have predecessor arcs with surviving witnesses (the predecessor coupling provides these, by induction around the ring). Therefore the greatest witness-closed subset survives.

Soundness (no phantom livelocks). At the fixed point, the surviving arcs satisfy cyclicity and backward closure. The surviving transitions form a witness-closed set. By Direction 2 of Theorem 5, $L^* \neq \emptyset$ implies a livelock exists. □

Theorem 8 (Complexity)**.** *Algorithm 1 terminates in at most $O(|T|^4)$ iterations, each running in $O(|T|^4)$ time. Total: $O(|T|^8)$ worst case.*

Proof. The product graph has $O(|T|^4)$ arcs. Each iteration removes at least one arc. SCC and the pruning checks are linear in the number of arcs. □

Remark 5 (Practical complexity)**.** *For self-disabling protocols, $|T| = O(m^2)$.*

This significantly improves the practical bounds. In the product graph, a node (t, w) requires $\mathrm{wr}(w) = \mathrm{pred}(t)$; for a fixed t, there are $O(|T|/m)$ valid witnesses w. The product graph therefore has $O(|T|^2/m) = O(m^3)$ nodes and $O(|T|^4/m^3) = O(m^5)$ arcs. Each iteration costs $O(m^5)$ for SCC computation.

In practice, convergence occurs in very few iterations—at most 6 across all 4,349 protocols tested with m up to 15—giving practical running time of $O(m^5)$. Pre-pruning T by removing transitions not on any H-cycle (an $O(m^4)$ step) further reduces the product graph size, often dramatically.

Remark 6 (Necessity of the product graph)**.** *A simpler algorithm operating directly on transitions and shadows—checking that each H-edge's shadow has an individual witness, without enforcing walk coherence—runs in $O(m^4)$ and is complete (never misses a livelock). However, it produces false positives: across 2,215 test protocols, it disagreed with the product graph on 55 cases (2.5%), reporting livelocks where none exist. The product graph eliminates these false*

positives by enforcing that witnesses form coherent closed walks, not merely that individual shadows are witnessable. The $O(m^5)$ practical cost of the product graph is the price of soundness.

6 The Circulation Law

The product graph determines *which* ring sizes support livelocks and how many processes are enabled in each.

6.1 Forward Enabling Map

Each simple cycle C in $H(T)$ induces a *forward target*: a closed walk whose predecessor values match C's written values. The *forward shift* measures the rotational offset between source and target:

$$s = \big(\text{canonical_zero}(C) - \text{canonical_zero}(\text{target})\big) \bmod N,$$

where canonical_zero is the position of the lexicographically minimum transition in the walk. For compound targets with repeated transitions, the minimum valid shift is used.

6.2 Closing Chains

Starting from a simple cycle C_0, repeated application of the forward map produces a chain:

$$C_0 \xrightarrow{s_0} C_1 \xrightarrow{s_1} C_2 \xrightarrow{s_2} \cdots$$

where each C_i is a closed walk (simple or compound) and s_i is the local forward shift. By finiteness, the chain closes: C_p is a rotation of C_0 for some period p.

6.3 The Law

Theorem 9 (Circulation Law)**.** *A livelock of a given class exists at ring size K if and only if K is a multiple of p and there exists e satisfying:*

$$e \equiv S(K) \pmod N, \quad 1 \le e \le K,$$

where $S(K) = \frac{K}{p} \cdot S(p)$, $S(p) = s_0 + s_1 + \cdots + s_{p-1}$, N is the walk length, and e is the number of enabled processes (tokens) in the livelock.

Different simple cycles with different forward maps, periods, and shifts produce different *livelock classes*. A protocol's complete livelock landscape is the set of all triples $(N, p, S(p))$ derived from its product graph.

Example 1. *The 3-coloring protocol $T = \{(0,0,1),(1,1,2),(2,2,0)\}$ has one simple cycle of length $N = 3$ with forward shift $s = 1$, period $p = 1$, and $S(K) = K$. The circulation law gives $e \equiv K \pmod 3$: for $K = 3$, $e = 3$ (all processes enabled); for $K = 4$, $e = 1$ (one token propagates).*

Example 2. *A protocol with 10 transitions (detailed in the appendix) has a chain of period $p = 3$ passing through two simple cycles and one compound walk, with forward shifts $s_0 = s_1 = s_2 = 1$ and $S(3) = 3$, $N = 4$. The circulation law gives $e \equiv 3K \pmod 4$: $K = 3$ yields $e = 3$ (verified by lockstep simulation), $K = 6$ yields $e = 2$ (4-token wave propagation with 144 valid configurations).*

7 Experimental Validation

The algorithm and all test protocols are available at `https://github.com/cosmoparadox/mathematical-tools`.

7.1 Known Protocols

Algorithm 1 correctly identifies:

- **Dijkstra's token ring** ($(1,1)$-asymmetric, $m = 3, 4, 5$): livelock, forward shift 1.
- **3-Coloring** (symmetric, $m = 3$): livelock, $e \equiv K \bmod 3$.
- **Sum-Not-2** (symmetric, $m = 3$): livelock-free.
- **Gouda–Haddix TB** (symmetric, $m = 8$): livelock, 14 surviving transitions out of 32, multiple livelock classes.

7.2 Randomized Stress Testing

We generated over 4,300 random self-disabling protocols across six categories: random seeds with $m \in \{3, \ldots, 10\}$, very dense protocols (near-complete transition sets), very sparse protocols (m to $2m$ transitions), structured adversarial (coloring and agreement variants with noise), large domain sizes $m \in \{11, \ldots, 15\}$ with targeted density, and compound-witness-like protocols (large m, few transitions). Transition counts ranged from 3 to 120.

For each protocol, we computed L^* via Algorithm 1 and cross-checked against exhaustive state-space search at small ring sizes ($K \leq 6$).

Results: Zero false positives. Zero false negatives across all 4,349 protocols tested. The algorithm converged in at most 6 iterations in every case.

7.3 Adversarial Protocols

Randomized testing exposed a protocol ($m = 8$, 13 transitions) where a prior approach based on individual shadow checking produced a false positive: every shadow had a witness, but no *coherent* witness walk existed. Shadow $(6, 5)$ was the bottleneck: no transition in T has $\mathrm{own} = 6$, $\mathrm{wr} = 5$. The product graph correctly identifies this protocol as livelock-free, demonstrating the necessity of the product graph over local shadow-by-shadow analysis.

7.4 K&E Adversarial Protocol

As a test of the product graph's faithfulness on structured adversarial inputs, we analyzed a protocol derived from Klinkhamer and Ebnenasir's SE tiling construction [16]: a symmetric self-disabling protocol with $m = 15$ and $|T| = 17$ transitions designed to exhibit complex livelock behavior. The product graph converges in a single iteration with all 17 transitions surviving ($|L^*| = 17$), producing 18 arcs and 8 simple H-cycles, of which 4 have forward targets in the product graph and 4 are dead-ends. The surviving cycles include two classes: length-3 cycles with shift 0, and length-6 cycles forming a closing chain of period $p = 3$ with total shift $S(3) = 9$, yielding livelocks at $K = 3, 6, 9, \ldots$ This demonstrates that the product graph captures the full livelock structure of protocols arising from tiling constructions.

8 Extension to $(1, 1)$-Asymmetric Protocols

For a $(1, 1)$-asymmetric ring with transition sets T_0 (distinguished process P_0) and T_o (all other processes), three product graphs capture the three coupling types:

1. $G_1 = G_\times(T_o, T_0)$: predecessor fires T_o, successor fires T_0. The $P_o \to P_0$ coupling.
2. $G_2 = G_\times(T_0, T_o)$: predecessor fires T_0, successor fires T_o. The $P_0 \to P_o$ coupling.
3. $G_3 = G_\times(T_o, T_o)$: both fire T_o. The bulk $P_o \to P_o$ coupling.

Each graph is built independently using the same four equivariance conditions.

8.1 Cross-Graph Backward Closure

In the symmetric case, the backward closure check verifies that each w-edge has a predecessor arc *within the same product graph*. For asymmetric protocols, the predecessor coupling may reside in a *different* product graph.

The three graphs form a ring of backward dependencies:

$$G_1 \to G_3 \to G_3 \to \cdots \to G_3 \to G_2 \to G_1$$

An arc in G_1 with w-pair (w_1, w_2) requires a predecessor arc with t-pair (w_1, w_2). Since the predecessor of P_0's predecessor fires T_o, this predecessor arc lies in G_3, not G_1. Similarly, arcs in G_2 require predecessor arcs in G_1, and arcs in G_3 require predecessor arcs in either G_3 (bulk) or G_2 (at the boundary).

8.2 Joint Algorithm

The algorithm extends naturally: apply SCC, square, and backward closure to each graph, but the backward closure check references the *appropriate neighboring graph* for the predecessor arc:

- For G_1 ($P_o \to P_0$): backward closure checks predecessor arcs in G_3.
- For G_2 ($P_0 \to P_o$): backward closure checks predecessor arcs in G_1.
- For G_3 ($P_o \to P_o$): backward closure checks predecessor arcs in G_3 itself (and in G_2 for the boundary process).

The square support S_t is computed per graph: $S_t^{(k)}$ is the set of t-components on cycles in G_k. The backward closure check for an arc in G_k with w-pair (w_1, w_2) verifies that the neighboring graph contains an arc $(w_1, u) \to (w_2, v)$ with $u, v \in S_t^{(\text{neighbor})}$.

Pruning in one graph may invalidate arcs in neighboring graphs through the cross-graph backward closure, triggering a cascade that propagates bidirectionally around the ring of graphs until quiescence.

8.3 Termination and Correctness

Theorem 10. *The $(1, 1)$-asymmetric algorithm terminates and is both sound and complete.*

Proof. **Termination.** The total number of arcs across all three graphs is finite, each iteration removes at least one, and arcs are never re-added.

Completeness. A livelock provides explicit walks and couplings that form surviving arcs in each graph. The cross-graph backward closure is satisfied because adjacent processes' walks serve as each other's predecessor couplings.

Soundness. The backward closure ensures that surviving cycles in each graph have realizable witness chains. The cross-graph backward closure connects these chains across the three coupling types, constructing a complete ring coupling. □

Remark 7. *For general (l, q)-asymmetry, one product graph per distinct coupling type suffices, with backward closure checks referencing the appropriate neighbor in the coupling ring. We leave the full treatment to future work.*

9 Related Work

Parameterized verification: general undecidability. Apt and Kozen [2] showed that verifying LTL properties for parameterized systems is Π_1^0-complete. Suzuki [18] strengthened this to unidirectional rings. Esparza [9] surveys the complexity landscape of parameterized verification. These results establish the general barrier; our work identifies a structural regime where decidability holds nonetheless.

Cutoff results. Emerson and Namjoshi [8] introduced cutoff-based reasoning for rings: if a property holds for rings up to a cutoff size, it holds for all sizes. Emerson and Kahlon [7] extended cutoffs to guarded protocols and ring-based message passing. These results reduce parameterized verification to finite model checking at a fixed size; our approach is complementary, reducing the problem to a fixed-point computation on the product graph independent of any specific ring size.

Well-quasi-orderings. Abdulla et al. [1] developed a general framework for decidability of infinite-state systems using well-quasi-orderings (WQOs): if states are well-quasi-ordered and transitions are monotonic, backward reachability terminates. Finkel and Schnoebelen [12] systematized this as the theory of well-structured transition systems (WSTSs). Our finiteness result is different in mechanism—it arises from the bounded column-type space of the equivariant product, not from a WQO on states—but follows the same pattern: lifting to a space where an ordering or finiteness property becomes explicit. Recent work extends WSTS techniques to threshold automata for round-based distributed algorithms [3].

Tiling and undecidability. Wang [19] introduced the domino problem: does a given tile set tile the plane? Berger [4] proved undecidability. The periodic variant—does the tile set admit a doubly-periodic tiling?—is also undecidable [14]. Klinkhamer and Ebnenasir [15] reduce the periodic domino problem to livelock detection, using an SE (South=East) restriction to enforce self-disabling. Our bounded witness theorem shows that the equivariant product space induced by self-disabling cannot faithfully encode the computational content of the periodic domino problem: the column-type finiteness forces a period bound that the domino problem's unbounded periods cannot survive.

Algebraic characterization and combinatorial lifting. Farahat [10] introduced an algebraic framework for livelock analysis in which livelocks are characterized by cyclic equivariance relations on the transition space—closed walks in the H-graph with compatible shadows—and derived a circulation law governing the relationship between ring size and number of enabled processes. Farahat and Ebnenasir [11] showed that the token-passing discipline induced by self-disabling enables local reasoning about global convergence properties. Klinkhamer and Ebnenasir [15] lifted Farahat's algebraic characterization to the combinatorial setting of torus tilings, establishing the equivalence between livelocks and doubly-periodic tilings via the "leads" relation—which is precisely the equivariance coupling. Their

formulation revealed the connection to the periodic domino problem, yielding Π^0_1-completeness. Our product transition graph completes the picture: it realizes the equivariance coupling as a finite combinatorial object, exposing the bounded witness property that their tiling framework left implicit.

Relationship to Klinkhamer and Ebnenasir's undecidability result. Their characterization [15] is correct and complete: a livelock exists if and only if m periodic propagations of period n exist, each leading the next. This characterization is realizable—every structure it identifies corresponds to a genuine livelock. The undecidability arises from the search over all periods n, which is *a priori* unbounded. Our work leverages a structural consequence of equivariance that yields a bound on this search: the column type (transition–witness pair) space has at most $|T|^2$ elements, bounding the tiling period independently of the ring size. This bounded witness property—absent from [15]—is the structural insight that yields decidability. We validate this on a protocol derived from their SE tiling construction [16] (Section 7).

Product constructions and finite abstractions. Our approach instantiates a pattern recurring across verification and combinatorics: problems that appear unbounded in their native representation become finite when lifted to a space where the governing invariant is explicit. The Myhill–Nerode theorem reduces language equivalence to a finite quotient; synchronized products in model checking compose automata to expose global reachability; symbolic dynamics lifts shift spaces to finite graphs of forbidden patterns. Our product transition graph plays an analogous role: it lifts equivariant transition walks to a finite graph of column types, making the bounded witness property explicit and computable.

Modern parameterized synthesis and verification. Recent approaches to parameterized verification and synthesis of self-stabilizing protocols, including SMT-based techniques [17], threshold automata [3], and cutoff-based methods [5], often yield semi-decision procedures or rely on abstraction and over-approximation. These methods are complementary: they approximate or restrict the search space to ensure termination in many cases, whereas our approach identifies a structural invariant that yields a finite representation and a complete decision procedure.

10 Conclusion

We have established that livelock detection for self-disabling unidirectional ring protocols is decidable, for all ring sizes simultaneously. The result rests on three contributions:

The product transition graph provides a faithful finite representation of the equivariance coupling between adjacent processes. Its nodes are transition–witness pairs in the zigzag, its arcs encode the four equivariance conditions, and it imposes no additional constraints. The finiteness of the node space ($|T|^2$) bounds the tiling period, collapsing the *a priori* unbounded search of [15] into a finite one.

The backward propagation lemma shows that backward closure at the arc level lifts to entire cycles: every cycle in a witness-closed product graph propagates backward to a new cycle of the same length. This is the core technical insight—the zigzag structure forces consecutive witnesses to chain coherently through a shared boundary value, even when drawn from different backward arcs.

The greatest fixed-point algorithm computes the largest witness-closed subset $L^* \subseteq T$ by monotone iteration on the product graph: repeatedly pruning arcs that violate cyclicity (SCC) or backward closure, until quiescence. The protocol admits a livelock if and only if $L^* \neq \emptyset$.

Future directions.

1. **Algebra of cycles.** The forward enabling map defines a structure on the cycle space of the product graph. The orbits, gluing operations, and shift homomorphisms may support a full algebraic classification of livelock classes.
2. (l, q)**-asymmetry.** Extending the three-graph construction to arbitrary numbers of process types. The structure scales to $l + q + 1$ product graphs.
3. **Livelock glossary.** Systematic enumeration and classification of livelock structures in classical ring protocols using the product graph and circulation law.
4. **Decidability boundary.** Characterizing which relaxations of self-disabling preserve decidability, and which cross into undecidability.

The structural lesson of this work is that the apparent unboundedness of transition-level livelock reasoning is an artifact of projection: in the equivariant product space, every valid behavior admits a bounded representation. Decidability emerges not from restricting the problem, but from choosing the space in which to reason about it.

A Detailed Examples

A.1 3-Coloring (Symmetric, $m = 3$)

$T = \{(0,0,1),\ (1,1,2),\ (2,2,0)\}$. All transitions survive. One simple cycle c_0 of length $N = 3$, self-enabling with forward shift $s = 1$.

Forward map: transition $(0,0,1)$ writes 1, enabling the successor's transition with $\mathrm{pred} = 1$ at index 1. Shift $= 1$ at every step.

Circulation law: $e \equiv K \pmod 3$. For $K = 2$: $e = 2$. For $K = 3$: $e = 3$ (all enabled). For $K = 4$: $e = 1$.

A.2 Dijkstra's Token Ring ($m = 3$, $(1,1)$-asymmetric)

$T_0 = \{(0,0,1),\ (1,1,2),\ (2,2,0)\}$, $T_o = \{(0,2,0),\ (1,0,1),\ (2,1,2)\}$.

Three product graphs: $G_\times(T_o, T_0)$, $G_\times(T_0, T_o)$, $G_\times(T_o, T_o)$. Forward shift 1 in T_0, shift 0 in T_o.

A.3 Compound Witness Protocol (Symmetric, $m = 16$)

$T = \{(5,2,3),\ (8,3,2),\ (2,7,9),\ (3,9,12),\ (2,12,15),\ (3,15,7),\ (7,5,8),\ (9,8,5),\ (12,5,8),\ (15,8,5)\}$.

All 10 transitions survive. Eight simple cycles in $H(T)$. Forward enabling chain: $c_0 \xrightarrow{s=1} c_3 \xrightarrow{s=1}$ compound $\xrightarrow{s=1} c_0$, period $p = 3$, $S(3) = 3$, $N = 4$.

Circulation law: $e \equiv 3K \pmod 4$. At $K = 3$: $e = 3$ (all processes enabled, verified by lockstep simulation with cycle length 4). At $K = 6$: $e = 2$ (4-token wave propagation, 144 verified configurations). Two distinct livelock classes coexist on the same protocol.

A.4 Trial 56: Shadow Coherence Failure ($m = 8$)

$$\begin{aligned} T = \{&(0,4,2),\ (0,7,6),\ (2,2,7),\ (2,5,7),\ (3,6,0),\ (4,0,2),\ (4,0,5),\\ &(5,2,0),\ (5,6,0),\ (6,3,4),\ (6,6,2),\ (7,0,6),\ (7,7,3)\} \end{aligned}$$

.

All 13 transitions lie on H-graph cycles. Of the 28 H-edges, 20 have individually witnessed shadows—only 8 lack witnesses. After removing those 8 edges, the remaining H-graph still contains cycles. A transition-level analysis (individual shadow checking) reports this protocol as a livelock.

However, no livelock exists. The unwitnessed shadows—including $(0,3)$, $(3,7)$, $(4,5)$, $(5,4)$, $(6,5)$, $(7,5)$—create gaps in the witness value space. Each H-cycle can find individual witnesses for its own shadows, but those witnesses form walks whose shadows hit the gaps. The incoherence manifests two levels deep in the witness chain.

The product graph $G_\times(T)$ starts with 42 arcs. Backward closure prunes 18 arcs in the first iteration; the surviving 24 arcs collapse to $\emptyset$ in the second (no remaining SCC). Result: livelock-free, confirmed by exhaustive state-space search at all ring sizes $K \leq 8$.

This protocol demonstrates that individual shadow witnessing is strictly weaker than the product graph's backward closure. The product graph detects that no coherent witness walk exists, even when individual shadows are witnessable.

References

[1] P. A. Abdulla, K. Čerāns, B. Jonsson, and Y.-K. Tsay. Algorithmic analysis of programs with well quasi-ordered domains. *Information and Computation*, 160(1–2):109–127, 2000.

[2] K. R. Apt and D. Kozen. Limits for automatic verification of finite-state concurrent systems. *Information Processing Letters*, 22(6):307–309, 1986.

[3] T. Baumeister, P. Eichler, S. Jacobs, M. Sakr, and M. Völp. Parameterized verification of round-based distributed algorithms via extended threshold automata. *arXiv preprint arXiv:2406.19880*, 2024.

[4] R. Berger. The undecidability of the domino problem. *Memoirs of the American Mathematical Society*, 66, 1966.

[5] R. Bloem, S. Jacobs, A. Khalimov, I. Konnov, S. Rubin, H. Veith, and J. Widder. *Decidability of Parameterized Verification*. Synthesis Lectures on Distributed Computing Theory. Morgan & Claypool, 2015.

[6] E. W. Dijkstra. Self-stabilizing systems in spite of distributed control. *Communications of the ACM*, 17(11):643–644, 1974.

[7] E. A. Emerson and V. Kahlon. Parameterized model checking of ring-based message passing systems. In *Proceedings of CSL*, LNCS 3210, pages 325–339. Springer, 2004.

[8] E. A. Emerson and K. S. Namjoshi. On reasoning about rings. *International Journal of Foundations of Computer Science*, 14(4):527–549, 2003.

[9] J. Esparza. Keeping a crowd safe: On the complexity of parameterized verification (invited talk). In *STACS*, LIPIcs vol. 25, pages 1–10. Schloss Dagstuhl, 2014.

[10] A. Farahat. *Automated Design of Self-Stabilization*. PhD thesis, Michigan Technological University, 2012.

[11] A. Farahat and A. Ebnenasir. Local reasoning for global convergence of parameterized rings. In *Proceedings of IEEE ICDCS*, pages 496–505, 2012.

[12] A. Finkel and Ph. Schnoebelen. Well-structured transition systems everywhere! *Theoretical Computer Science*, 256(1–2):63–92, 2001.

[13] M. G. Gouda and F. Haddix. The alternator. In *Proceedings of the 3rd Workshop on Self-Stabilizing Systems*, 1997.

[14] Y. Gurevich and I. Koryakov. Remarks on Berger's paper on the domino problem. *Siberian Mathematical Journal*, 13(2):319–321, 1972.

[15] A. Klinkhamer and A. Ebnenasir. On the verification of livelock-freedom and self-stabilization on parameterized rings. *ACM Transactions on Computational Logic*, 20(3):16:1–16:36, 2019.

[16] A. Ebnenasir and A. Klinkhamer. On the verification of livelock-freedom and self-stabilization on parameterized rings. Computer Science Technical Report CS-TR-19-01, Michigan Technological University, January 2019.

[17] N. Mirzaie, F. Faghih, S. Jacobs, and B. Bonakdarpour. Parameterized synthesis of self-stabilizing protocols in symmetric networks. *Acta Informatica*, 57(1–2):271–304, 2020.

[18] I. Suzuki. Proving properties of a ring of finite-state machines. *Information Processing Letters*, 28(4):213–214, 1988.

[19] H. Wang. Proving theorems by pattern recognition—II. *Bell System Technical Journal*, 40(1):1–41, 1961.